# OSGuard

## Risk-Aware Operating-System Update Orchestration

*An explainable OS-update risk assessment and rollout simulation platform – educational project*

| Field | Entry |
|---|---|
| Team name | NOVA |
| Team members | Anirudh Koppuravuri – 24BCE0490<br>Kalivarapu Prabhas – 24BCE2279<br>Barath Sornaraj – 24BCE2599 |
| Institution | Vellore Institute of Technology |
| Course / course code | [EDITABLE: course name and code] |
| Lecturer / evaluator | [EDITABLE: lecturer name] |
| Slot / semester | [EDITABLE] |
| Submission date | [EDITABLE: dd Month yyyy] |
| Video link | [EDITABLE: add after upload – see Section 14] |

*Academic prototype. All machines, patches, vulnerabilities and outcomes are SYNTHETIC; results are simulation outputs, not real-world measurements. OSGuard never modifies the host operating system, installs patches or runs privileged operations. OSGuard is not xv6, not a kernel and not a real multi-OS patching agent. Developed for educational purposes (coursework) only.*

# Table of contents

1. Submission requirements and traceability ..... 4
2. Abstract, objectives and scope ..... 5
2.1 Abstract ..... 5
2.2 Problem statement ..... 5
2.3 Objectives ..... 5
2.4 Scope and contributions ..... 5
3. Operating-system context and concepts ..... 5
3.1 Host and runtime environment (read from the machine) ..... 5
3.2 The simulated fleet ..... 6
3.3 Operating-systems concepts demonstrated ..... 6
3.4 What OSGuard is not ..... 6
4. Industry problem analysis and existing solutions ..... 6
4.1 OSGuard's defensible contribution ..... 7
5. System design ..... 7
5.1 Architecture and data flow ..... 7
5.2 Components ..... 7
5.3 Database structure ..... 8
5.4 API design ..... 8
5.5 Technology stack (installed versions) ..... 8
6. Algorithms and safety gates ..... 9
6.1 Urgency score ..... 9
6.2 Compatibility risk and uncertainty ..... 9
6.3 Readiness rules and safety gates ..... 9
6.4 Rollout state transitions ..... 10
6.5 Worked examples (from the running application, seed 2026) ..... 10
6.6 Failure model (hidden ground truth of the simulation) ..... 10
6.7 Metrics ..... 10
7. Implementation verification ..... 11
7.1 Commands executed and results ..... 11
7.2 Evidence that unsafe states cannot silently pass ..... 11
7.3 Defect found and fixed ..... 12
8. Experiments ..... 12
8.1 Methodology and reproducible configuration ..... 12
8.2 Results (simulation output) ..... 12
8.3 Limitations ..... 13

9. Native debugging with GDB ..... 13
9.1 Applicability ..... 13
9.2 Build and cross-check ..... 13
9.3 GDB session (real output, GDB 17.1, WSL2 Ubuntu 26.04) ..... 13
9.4 What the session shows ..... 14
10. Screenshots (captured from the running application) ..... 15
11. Viva questions and answers ..... 18
12. Conclusion and future work ..... 19
13. References ..... 19
14. Submission checklist and video package ..... 19
Appendix A. Source listings ..... 21
Appendix B. Start-up commands (Windows) ..... 22

# 1. Submission requirements and traceability

Assignment title as given: “Mini Project: Modifying a Small OS Kernel”. OSGuard is an OS-update orchestration simulator for educational purposes; it does not modify an operating-system kernel (see Section 3.4). The detailed rubric (“See Rubrics”) was not available when this report was prepared, so the rows below follow the written submission instructions. Status values are conservative: an item is only “Complete” if it was verified by a command or file listed here.

*Table 1 – Requirements traceability*

| # | Requirement | Implementation / location | Evidence | Status |
|---|---|---|---|---|
| 1 | Single PDF with all work | This document (DOCX + PDF built from the same source) | docs/OSGuard_Project_Report.pdf (main) and docs/OSGuard_Project_Report_full_listings.pdf | Complete |
| 2 | Answers to questions | Section 11 (viva Q&A). The assignment’s own question list was not supplied | Section 11 | Partly – team to add the lecturer’s exact questions |
| 3 | Code listings | Section 9 and Appendix A excerpts; full listings are in OSGuard_Project_Report_full_listings.pdf | Files hashed at build time | Complete |
| 4 | GDB session logs | Native diagnostic harness native/rollout_gate.c debugged in GDB 17.1 (Section 9) | native/gdb_session.log, Section 9 | Complete for the C harness only; Python/TypeScript are not GDB-debugged |
| 5 | Screenshots | Section 10: dashboard, fleet, risk assessment, rollout success, fault containment, comparison, analytics, audit, sources | docs/screenshots/*.png (captured from the running app) | Complete |
| 6 | Modify a small OS kernel (assignment title) | Not implemented: OSGuard is a simulator running on Windows 11, not a kernel and not xv6 | Section 3 | NOT satisfied by this project |
| 7 | Screen-recorded walkthrough; every member explains a portion, runs the program, own voice | docs/VIDEO_WALKTHROUGH_SCRIPT.md and docs/scripts/ (one script per member, sequential) | None yet – must be recorded by the team | Unresolved – team action |
| 8 | Video on Clipchamp/Google Drive (shared with the evaluator) or YouTube Unlisted | Checklist in Section 14 | — | Unresolved – team action |
| 9 | List mapping each member to a timestamp range | Template in Section 14 with member names | — | Template only – fill after recording |
| 10 | Readiness for follow-up questions | Section 11 and the scripts’ Q&A | — | Prepared; team must study it |
| 11 | Working implementation: risk engine, blockers, fleet, rollout simulator, thresholds, fault injection, audit, experiments, APIs, UI | Sections 5–8 | 56 backend tests, 12 frontend tests, type-check and production build all passed (Section 7) | Complete |

# 2. Abstract, objectives and scope

## 2.1 Abstract

Operating-system updates close security holes but also break drivers, applications and services. OSGuard is a prototype that decides, for each machine in a synthetic fleet, whether an update is urgent, whether it is safe enough to deploy, and how to roll it out in stages so that a bad update is contained. It separates three questions that are often blended into one score: how urgent is the patch (severity, exposure, criticality, age), what evidence says it will fail (failed compatibility checks), and what evidence is missing (unknown checks, absent telemetry). Missing or failed mandatory evidence always blocks deployment. Eligible machines are updated through a canary and staged batches with health thresholds and automatic halting, every decision is written to an audit log in SQLite, and reproducible experiments compare the staged strategy with an all-at-once baseline.

## 2.2 Problem statement

How can an organisation prioritise security patches while reducing compatibility risk and service disruption, using evidence it can inspect and explain, and how can that policy be evaluated without risking real systems?

## 2.3 Objectives

- Compute an explainable urgency score from security severity, exposure, operational criticality and patch age with configurable weights.
- Track compatibility risk (failed evidence) and uncertainty (missing evidence) separately, and never treat missing or failed mandatory evidence as safe.
- Simulate canary and staged rollouts with latency/error thresholds, fault injection and automatic halting, recording each decision and its reason.
- Compare staged and all-at-once strategies reproducibly on blast radius, completion time, simulated exposure, unnecessary delays and safety violations.
- Persist decisions, rollout events, triggered rules and experiment results, and allow filtering and export.

## 2.4 Scope and contributions

In scope: a deterministic synthetic fleet of 120 machines, a rule-based risk engine, a simulator, six scenarios, a React/TypeScript interface with an interactive 3D fleet map, and a native C diagnostic harness used for GDB debugging. Out of scope: real patch installation, real vendor telemetry, machine learning, authentication, multi-user operation.

- A transparent readiness gate (rules B1–B6) that keeps negative evidence and missing evidence distinct.
- A resumable rollout state machine with explicit allowed transitions and an auditable halt decision.
- A paired-seed comparison method in which both strategies face identical hidden outcomes.
- A cross-check of the safety rules in an independent native implementation (0 mismatches over 120 machines, Section 9).

# 3. Operating-system context and concepts

## 3.1 Host and runtime environment (read from the machine)

*Table 2 – Execution environment*

| Item | Value |
|---|---|
| Host OS | Microsoft Windows 11 Home Single Language, version 10.0.26300 (build 26300), 64-bit (AMD64) |
| CPU / memory | 13th Gen Intel Core i5-13450HX / 15.7 GB |

| Item | Value |
|---|---|
| Python / Node.js / npm | 3.14.7 / v22.13.1 / 10.9.2 |
| SQLite library (bundled with Python) | 3.50.4 |
| Shells | PowerShell 5.1, Git Bash |
| WSL distribution used only for the C/GDB demonstration | WSL2 Ubuntu 26.04 LTS, kernel 6.18.33.2-microsoft-standard-WSL2 |
| GDB / GCC (inside WSL) | GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1 / gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0 |

OSGuard itself (Python/FastAPI backend and React/TypeScript frontend) ran natively on Windows 11. Only the diagnostic harness in Section 9 was compiled and debugged in the WSL2 Ubuntu environment.

### 3.2 The simulated fleet

The fleet contains 120 synthetic machine records. Each carries an OS family and version as a text label: Windows Server (2019, 2022, 2025), Windows 11 (23H2, 24H2), Ubuntu (22.04, 24.04), RHEL (8.10, 9.4), Amazon Linux (2023), macOS (14, 15) and ICS/Embedded (RTOS 4.2, HMI 7.1). These are not independently running operating systems: no guest OS is installed, booted, emulated or patched. Patch identifiers (for example SYN-WS-2026-10), CVSS-style scores and the "vendor hold" are invented.

### 3.3 Operating-systems concepts demonstrated

- **Patch and update management:** urgency from severity, exposure, criticality and age; restart-required versus live-patchable updates (the live-patch idea follows AWS's documented Amazon Linux 2023 kernel live patching, Section 4).
- **Compatibility and dependency risk:** driver, application and backup evidence gate each update; a safeguard-hold style block models vendor-known incompatibility.
- **Reliability and recovery:** health thresholds, automatic halt, rollback of unhealthy machines, and a rollout state machine (planned → running → completed | halted | aborted).
- **Concurrency:** the web server runs handlers on worker threads; an intermittent defect where an SQLite connection crossed threads was found and fixed (Section 7.3).
- **Native debugging:** stack frames, conditional breakpoints, watchpoints and return-value inspection on a C program with GDB (Section 9).

### 3.4 What OSGuard is not

- Not xv6, not a teaching kernel, not QEMU-based.
- Not a real patching agent and not a replacement for commercial tools.
- Not using machine learning or AI; all logic is deterministic rules and arithmetic.
- Not tied to real companies' data: all numbers are synthetic or simulated.

## 4. Industry problem analysis and existing solutions

On 26 October 2026 the text of each page below was fetched and the statements were checked against it. Linked sub-pages and PDFs were not read; the Apple and CISA entries rest on an index or table of contents only. Entries marked "Our observation" are the team's own comparison, not vendor claims.

*Table 3 – Existing solutions checked against the cited pages*

| Platform | What the cited page says | Limitation / our observation |
|---|---|---|

| Platform | What the cited page says | Limitation / our observation |
|---|---|---|
| Microsoft Windows – safeguard holds | Microsoft uses quality and compatibility data to identify issues that could cause a feature update to fail or roll back, and may hold the update for affected devices until a fix is verified. Windows Update for Business reports list active holds; Autopatch-managed devices also benefit. | Holds only affect devices using the Windows Update service; admins using other channels (media, WSUS) must watch known issues themselves. Our observation: holds reflect vendor-known issues, not your own application inventory. |
| AWS Systems Manager Patch Manager | Patch baselines per supported OS type; patch policies, maintenance windows or on-demand “Patch now”; compliance reporting. | Our observation: the page describes policy and scheduling, not per-machine compatibility-evidence scoring. |
| AWS – Amazon Linux 2023 kernel live patching | Applies specific security and critical-bug kernel patches to a running kernel without rebooting; can improve availability until reboot. | Some issues cannot be fixed by live patching and are only shipped in full kernel updates; the page lists functional limitations while a live patch is applied. |
| Google Cloud – GKE upgrade best practices | Covers release channels, maintenance windows and exclusions, surge and blue-green node upgrade strategies, Pod Disruption Budgets, version-skew limits, and testing changes in environments separate from production. | Scope is GKE clusters, not arbitrary fleets. Our observation: workload-specific go/no-go decisions are left to the operator. |
| Apple Platform Deployment | The guide has a “Manage software updates” section: about software updates, the update process, installing and enforcing updates, and testing updates with the AppleSeed for IT beta programme. | Only the table of contents was read; detailed capabilities are on sub-pages not reviewed. |
| CISA – ICS Recommended Practices | The page indexes ICS cybersecurity documents, including “Recommended Practice for Patch Management of Control Systems” and “Improving ICS Cybersecurity with Defense-in-Depth Strategies”. | Only the index was read; the PDFs were not reviewed. These are guidance documents, not an orchestration tool. |

### 4.1 OSGuard’s defensible contribution

OSGuard does not claim to outperform any of these products and does not replace them. Its contribution is smaller and specific: it is a transparent, inspectable model of an evidence-gated rollout in which urgency, failed evidence and missing evidence are kept separate, every recommendation lists the rules behind it, and the safety policy can be exercised against reproducible faults without touching real systems. It is a teaching and evaluation prototype.

## 5. System design

### 5.1 Architecture and data flow

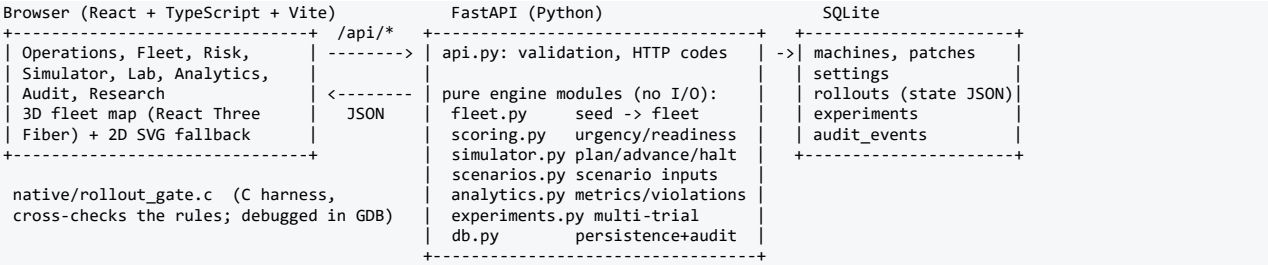



*Figure 1 – Architecture of OSGuard (text diagram)*

Data flow for a staged rollout: (1) the UI posts a rollout request; (2) the API loads machines and the scoring configuration from SQLite; (3) a scenario, if chosen, rewrites a deep copy of the fleet; (4) the engine assesses every machine and plans batches; (5) each “advance” runs one batch and returns events and metrics; (6) the full rollout state and any new audit events are saved; (7) the UI renders batches, metrics and decisions.

### 5.2 Components

*Table 4 – Component responsibilities*

| Component | Responsibility |
|---|---|

| Component | Responsibility |
|---|---|
| fleet.py | Deterministic generator of 120 machines and the synthetic patch catalogue from a seed (default 2026) |
| scoring.py | Urgency score, compatibility risk, uncertainty, readiness rules B1–B6, recommended action with explanations |
| simulator.py | Failure model, planning (eligibility, ordering, canary choice), batch execution, health evaluation, halt, abort, state machine |
| scenarios.py / experiments.py | Six reproducible scenarios; multi-trial paired comparison of both strategies |
| analytics.py | Blast radius, completion time, exposure, unnecessary delays, safety-rule violations, timelines |
| db.py | SQLite schema, persistence, audit log, CSV export formatting |
| api.py | FastAPI routes, validation and error mapping (404 / 409 / 422) |
| frontend/src | Eight views, the 3D fleet map (Fleet3D.tsx, instanced meshes, pointer-reactive), 2D fallback, typed API client, validation helpers |
| native/ | C diagnostic harness, CSV exporter, GDB command script and captured session log |

## 5.3 Database structure

*Table 5 – SQLite schema*

| Table | Columns / content |
|---|---|
| machines, patches | id; data (JSON document of the synthetic record) |
| settings | key; value (scoring configuration, fleet seed and size) |
| rollouts | id; created_at; updated_at; strategy; scenario_id; status; state (full serialised rollout state) |
| experiments | id; created_at; scenario_id; seed; trials; result (summary, representative run, per-trial metrics) |
| audit_events | id; ts (UTC); category; action; subject; rule; detail (JSON); rollout_id; experiment_id; sim_minute |

## 5.4 API design

*Table 6 – HTTP API*

| Method | Path | Purpose |
|---|---|---|
| GET | /api/health, /api/dashboard | Status; dashboard aggregates |
| GET | /api/fleet, /api/fleet/{id}, /api/fleet/facets | Search, filter, sort; one machine with full explanation |
| POST | /api/fleet/reseed | Regenerate the synthetic fleet from a seed (audited) |
| GET/PUT/POST | /api/config/scoring (+ /reset) | Read, update (validated, audited), reset scoring configuration |
| GET | /api/patches, /api/scenarios, /api/sources | Patch catalogue; scenarios; research sources |
| POST | /api/rollouts, /{id}/advance, /{id}/abort | Create, step and abort a rollout (invalid transitions return 409) |
| GET | /api/rollouts, /api/rollouts/{id} | List; details with metrics and timelines |
| POST/GET | /api/experiments, /api/experiments/{id} | Run and read multi-trial experiments |
| GET | /api/analytics | Latest experiment per scenario |
| GET | /api/audit, /api/audit/export?format=csv\|json | Filtered audit log; export |

## 5.5 Technology stack (installed versions)

*Table 7 – Technology stack*

| Layer | Technology | Version |
|---|---|---|
| Backend | Python / FastAPI / Starlette / Pydantic / Uvicorn | 3.14.7 / 0.143.3 / 1.8.0 / 2.14.1 / 0.54.0 |

| Layer | Technology | Version |
|---|---|---|
| Backend tests | pytest / httpx | 9.1.1 / 0.28.1 |
| Frontend | React / TypeScript / Vite / Tailwind CSS / Framer Motion | 18.3.1 / 5.6.3 / 5.4.11 / 3.4.15 / 11.11.17 |
| 3D and charts | Three.js / @react-three/fiber / Recharts | 0.169.0 / 8.17.10 / 2.13.3 |
| Frontend tests | Vitest | 2.1.5 |
| Native demo | GCC / GDB (WSL2 Ubuntu 26.04) | 15.2.0 / 17.1 |

All libraries are open source with permissive licences (MIT, BSD-3-Clause for Uvicorn). The code, copy and diagrams are original to this project.

# 6. Algorithms and safety gates

## 6.1 Urgency score

urgency = $100 \times \Sigma (w_i \times factor_i)$ with the four weights normalised to sum to 1. Defaults and factor definitions:

*Table 8 – Urgency factors*

| Factor | Value in 0..1 | Default weight |
|---|---|---|
| Severity | min(1, CVSS/10 + 0.15 if known-exploited) | 0.40 |
| Exposure | internet 1.0, internal 0.5, isolated 0.2 | 0.25 |
| Criticality | (criticality − 1) / 4 | 0.15 |
| Patch age | min(1, days since fix released / 60) | 0.20 |

Bands (editable): critical ≥ 70, high ≥ 55, medium ≥ 35, otherwise low.

## 6.2 Compatibility risk and uncertainty

Mandatory checks are driver_compat, app_compat and backup_verified (weight 1.0 each); optional checks are agent_compat and disk_space (0.5 each); telemetry weighs 1.5. Compatibility risk = weight of checks that failed ÷ weight of all checks (excluding telemetry). Uncertainty = weight of unknown checks (plus telemetry weight if absent) ÷ total weight. A missing result is never counted as a pass.

## 6.3 Readiness rules and safety gates

*Table 9 – Readiness rules*

| Code | Rule | Effect |
|---|---|---|
| B1 | A mandatory check failed | negative evidence → blocked |
| B2 | A mandatory check has no result | missing evidence → insufficient evidence |
| B3 | No health telemetry (policy default: required) | missing evidence → insufficient evidence |
| B4 | Vendor/safeguard hold for this OS version | negative evidence → blocked |
| B5 | Pre-update latency ratio > 1.5 or error rate > 3% | negative evidence → blocked |
| B6 | OS family needs manual approval (default ICS/Embedded) | needs approval |
| W1–W3 | Optional check failed/unknown; telemetry optional but missing | warning → ready with caution |

Precedence: blocked > insufficient evidence > needs approval > ready with caution > ready. Unsafe compatibility states (failed mandatory check, vendor hold, degraded health) and missing mandatory evidence or telemetry therefore cannot yield “ready”; they defer the machine from the staged rollout. If an operator

disables the telemetry requirement, staged rollouts still fail closed: a machine whose health cannot be verified during the soak is counted unhealthy.

## 6.4 Rollout state transitions

*Table 10 – Rollout state machine*

| From | Allowed next states | Triggered by |
|---|---|---|
| planned | running, aborted | advance (first batch) / abort |
| running | running, completed, halted, aborted | advance: next batch passes / last batch passes / breach / abort |
| completed, halted, aborted | none (terminal) | any further advance or abort returns HTTP 409 |

Staged execution per batch: deploy → soak → health evaluation. A machine is unhealthy if unreachable, latency/baseline > 1.5, or error rate > 3%. The rollout halts if the canary has any unhealthy machine or if a batch's unhealthy fraction exceeds 10%. Unhealthy machines are rolled back and later batches are marked skipped and never touched. The baseline deploys to every in-scope machine at once with no gate, no soak and no halt.

## 6.5 Worked examples (from the running application, seed 2026)

**Example A – urgent and deployable (vm-081, Windows Server 2022, internet-facing, criticality 5).** Severity: CVSS 8.1 → 0.81 + 0.15 exploited bonus = 0.96 × 0.40 × 100 = 38.4. Exposure: 1.0 × 0.25 × 100 = 25.0. Criticality: (5−1)/4 = 1.0 × 0.15 × 100 = 15.0. Age: 14/60 = 0.233 × 0.20 × 100 = 4.7. Urgency = 83.1 (critical). All mandatory checks pass and telemetry is present, so readiness is Ready and the action is "deploy – expedited canary" (Figure 3).

**Example B – blocked (vm-003, macOS 15, internal, criticality 2).** Urgency = 47.45, displayed as 47.5 (severity 21.2 + exposure 12.5 + criticality 3.75 + age 10.0). The driver_compat check failed (rule B1), so compatibility risk = 1.0 / 4.0 = 25% (failed weight 1.0 over risk weight 3×1.0 + 2×0.5) and uncertainty = 0%. Readiness is Blocked and the action is "resolve blockers" (Figure 4).

**Example C – automatic halt (rollout #9, injected bad build, seed 2026).** Canary (2 machines) passed with 0 unhealthy and Stage 1 passed with 1 of 10 unhealthy (10%, which is not greater than the 10% tolerance). Stage 2 had 3 of 17 unhealthy = 17.6%, shown as 18%, which exceeds 10%, so the rollout halted; the 19 machines of Stage 3 were never touched (Figures 6–7 and the audit record in Figure 9).

## 6.6 Failure model (hidden ground truth of the simulation)

Independent probabilities combine as $p = 1 - \Pi(1 - p_i)$, capped at 0.97: background 0.01; failed driver check 0.55; failed application check 0.45; each unknown mandatory check 0.08; already degraded 0.30; vendor hold 0.50; plus any injected fault. A machine fails if a SHA-256-derived uniform value for (seed, machine, patch) is below p, so both strategies face identical outcomes for a given seed. 15% of non-boot failures are "subtle regressions" that remain under the default thresholds and are counted as undetected degradations.

## 6.7 Metrics

*Table 11 – Experiment metrics*

| Metric | Definition |
|---|---|
| Failure blast radius | Deployed machines whose hidden outcome is failure (also reported criticality-weighted) |
| Completion time | Simulated minutes until the rollout stopped, including rollbacks; plus the fraction of the fleet patched |
| Simulated unpatched exposure | Σ (urgency/100) × hours unprotected over a 168-hour horizon |

| Metric | Definition |
|---|---|
| Unnecessary delays | Machines deferred by the gate or skipped after a halt whose hidden outcome would have been success |
| Safety-rule violations | Deployments to a machine with an unmet blocker (B1–B6), a criticality ≥ 4 machine in the canary, or a breach with no ability to halt |

# 7. Implementation verification

## 7.1 Commands executed and results

*Table 12 – Verification commands*

| Check | Command (Windows) | Result |
|---|---|---|
| Backend tests | cd backend; .\.venv\Scripts\python -m pytest -v | 56 passed |
| Frontend tests | cd frontend; npm test | 12 passed |
| Type-check and production build | cd frontend; npm run build | succeeded |
| Native cross-check | gcc -g -O0 -Wall -Wextra rollout_gate.c; ./rollout_gate fleet_export.csv (WSL) | 120 machines, 0 mismatches |

*Table 13 – pytest results per file*

| Backend test file | Tests passed |
|---|---|
| test_experiments_api.py | 22 |
| test_fleet.py | 3 |
| test_scoring.py | 14 |
| test_simulator.py | 17 |
| Total | 56 |

## 7.2 Evidence that unsafe states cannot silently pass

*Table 14 – Safety properties and their tests*

| Property | Test (backend/tests) |
|---|---|
| Failed mandatory check is never ready | test_failed_mandatory_check_blocks_and_never_ready |
| Missing mandatory evidence is not treated as safe | test_missing_mandatory_evidence_is_not_silently_safe |
| Missing telemetry blocks | test_missing_telemetry_blocks; test_default_policy_defers_machine_without_telemetry |
| Fail-closed without telemetry when the requirement is relaxed | test_missing_telemetry_fails_closed_unless_disabled |
| Negative evidence outranks missing evidence | test_negative_evidence_outranks_missing_evidence |
| Canary failure halts and later batches are untouched | test_canary_failure_halts_and_skips_later_batches |
| Later-batch fraction above tolerance halts | test_later_batch_halts_when_fraction_exceeds_tolerance |
| Staged strategy has zero rule violations; baseline does not | test_baseline_ignores_gates_staged_has_zero_violations |
| Invalid transitions rejected | test_batch_progression_and_state_machine; test_abort_planned_rollout_is_allowed_and_terminal |
| Audit records exist and are not duplicated | test_rollout_lifecycle_and_audit; test_database_persistence_across_restart |

### 7.3 Defect found and fixed

During a launcher run the log showed an intermittent HTTP 500 on /api/fleet: “SQLite objects created in a thread can only be used in that same thread”. The per-request connection was opened in one worker thread and used in another. The connection is now opened with check_same_thread=False (each request owns its connection and uses it sequentially). The regression test test_concurrent_requests_do_not_trip_sqlite_thread_affinity fails without the fix and passes with it. The algorithms were not changed.

## 8. Experiments

### 8.1 Methodology and reproducible configuration

- Fleet: generate_fleet(seed 2026, 120 machines). Scoring: default ScoringConfig. Trials: 10 outcome seeds per scenario (2026–2035).
- Rollout defaults: batches 5/20/35/40 %, soak 30 min, deploy base 5 min + 4 min per wave of 20 machines, max latency ratio 1.5, max error rate 3%, canary strict, batch tolerance 10%, rollback 20 min, baseline manual detection 60 min, horizon 168 h, background failure probability 0.01.
- For each scenario and trial, both strategies run on the same fleet copy and the same hidden outcomes; metrics are averaged (mean) across trials.
- Reproduce: Experiment lab → Run all scenarios (seed 2026, 10 trials), or POST /api/experiments with {scenario_id, seed: 2026, trials: 10}. Results are deterministic.

*Table 15 – Scenario definitions*

| Scenario | Fleet rewrite / overrides |
|---|---|
| Healthy update | All checks pass, telemetry on, latency at baseline, vendor holds removed; background failure probability 0 |
| Compatibility failures | 35% of Windows Server and Ubuntu hosts fail driver_compat; 20% of database/api hosts fail app_compat |
| Critical vulnerability | Every patch CVSS ≥ 9.8 and known-exploited; internet-facing hosts criticality ≥ 4; soak shortened to 10 min |
| Missing telemetry | 30% of machines lose telemetry; half of those also lose app_compat; background failure probability 0.12 |
| Degraded application health | 25% of machines already at 1.8–3.0× baseline latency and 4–9% errors |
| Injected fault (bad build) | 60% of Windows Server hosts hit an error-spike regression regardless of recorded evidence |

### 8.2 Results (simulation output)

*Table 16 – Means over 10 trials, shown as staged / all-at-once*

| Scenario | Blast radius | Safety viol. | Unneeded delays | Completion (min) | Exposure (urg·h) | Fleet patched | Staged halts |
|---|---|---|---|---|---|---|---|
| Healthy update | 0.0 / 0.0 | 0 / 8 | 8.0 / 0.0 | 164 / 29 | 1039 / 18 | 92% / 100% | 0/10 |
| Compatibility failures | 0.4 / 25.6 | 0 / 95 | 39.8 / 0.0 | 164 / 109 | 6888 / 3184 | 36% / 70% | 0/10 |
| Critical vulnerability | 0.4 / 17.2 | 0 / 80 | 37.2 / 0.0 | 84 / 109 | 6408 / 2633 | 47% / 78% | 0/10 |
| Missing telemetry | 1.8 / 28.7 | 0 / 132 | 63.9 / 0.0 | 114 / 109 | 9425 / 3248 | 10% / 69% | 8/10 |
| Degraded health | 0.3 / 24.3 | 0 / 102 | 41.0 / 0.0 | 162 / 109 | 6662 / 4007 | 36% / 60% | 0/10 |
| Injected fault | 4.6 / 30.3 | 0 / 80 | 45.4 / 0.0 | 157 / 109 | 7986 / 3521 | 26% / 68% | 10/10 |

- The staged strategy has a far smaller blast radius and zero safety-rule violations in every scenario; the baseline ignores the rules by design.
- The staged strategy also has higher simulated exposure, lower fleet coverage and many unnecessary delays. This is the cost of gating. In the healthy scenario the 8 delayed machines are the industrial-system

machines held by the approval gate (B6); this was checked directly (102 pending machines: 94 ready, 8 needs-approval, all ICS/Embedded).

- In the injected-fault scenario the staged blast radius is 4.6, not zero: the fault is not predictable from recorded evidence, so containment depends on the canary and the halt rule. Staged rollouts halted in 10/10 trials.

### 8.3 Limitations

- The failure model makes failures more likely on machines with failed or missing evidence, so evidence gating helps by construction. The injected-fault and subtle-regression cases show limits but do not remove this circularity.
- Time is a coarse simulated clock; no networks, dependencies, maintenance windows, reboot ordering or retries after rollback are modelled.
- Vendor holds, CVSS values and patch identifiers are invented; no real company incidents or telemetry are used.
- Results describe the mechanism and its trade-offs, not real-world improvement.

## 9. Native debugging with GDB

### 9.1 Applicability

GDB debugs native executables. OSGuard's application code is Python and TypeScript, so GDB does not apply to it and no claim is made that it was debugged with GDB. To give the safety rules a genuine native debugging demonstration, the team wrote a small, clearly labelled C diagnostic harness, native/rollout_gate.c. It is not part of the application, not a kernel and not xv6. It re-implements (a) the readiness gate (rules B1–B6, default configuration) and (b) the automatic-halt decision, reads the exported fleet (native/fleet_export.csv, written by native/export_fleet.py from the Python engine), recomputes every verdict and compares it with the Python result.

### 9.2 Build and cross-check

```
# Windows (PowerShell), from backend/ : export fleet and Python verdicts
$env:PYTHONPATH = (Get-Location).Path
.\.venv\Scripts\python ..\native\export_fleet.py ..\native\fleet_export.csv
# WSL Ubuntu : compile with debug symbols and run
gcc -g -O0 -Wall -Wextra -o /tmp/rollout_gate rollout_gate.c
/tmp/rollout_gate fleet_export.csv
# result: machines=120 mismatches=0  (ready 42, ready_with_caution 6, needs_approval 4,
#                                     insufficient_evidence 14, blocked 36, compliant 18)
```

The native counts equal the Python engine's counts for the default fleet; the compilation produced no warnings under -Wall -Wextra.

### 9.3 GDB session (real output, GDB 17.1, WSL2 Ubuntu 26.04)

Command: gdb -q -batch -x gdb_session.gdb --args /tmp/rollout_gate fleet_export.csv. The transcript below is the unedited captured output (native/gdb_session.log).

```
### 1. Conditional breakpoint on the readiness gate for machine vm-003
Breakpoint 1 at 0x133a: file rollout_gate.c, line 49.


This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".


Breakpoint 1, assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:49
49    {


### 2. Backtrace and inspect the arguments
#0  assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:49
```

```
#1  0x0000555555555a5b in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:131
$1 = {
  id = "vm-003\000\000\000\000\000\000\000\000\000",
  family = "macOS", '\000' <repeats 18 times>,
  version = "15", '\000' <repeats 13 times>,
  expected = "blocked", '\000' <repeats 24 times>,
  driver = CHK_FAIL,
  app = CHK_PASS,
  backup = CHK_PASS,
  agent = CHK_PASS,
  disk = CHK_PASS,
  telemetry = 1,
  vendor_hold = 0,
  compliant = 0,
  latency_ratio = 0.99921099999999996,
  error_rate = 0.0067999999999999996
}
$2 = {
  require_telemetry = 1,
  latency_ratio_max = 1.5,
  error_rate_max = 0.029999999999999999,
  max_unhealthy_fraction = 0.10000000000000001,
  canary_halt_on_any_failure = 1
}

### 3. Watch the blocker bit mask while the rules execute
Hardware watchpoint 2: blockers


Hardware watchpoint 2: blockers


Old value = 2
New value = 0
assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:50
50        int blockers = 0, negative = 0, missing = 0, warnings = 0;
$3 = 0
$4 = 0


Hardware watchpoint 2: blockers


Old value = 0
New value = 1
0x00005555555553bd in assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at
rollout_gate.c:57
57            if (mandatory[i] == CHK_FAIL)         { blockers |= B1; negative = 1; }


### 4. Finish the function and read the verdict


Watchpoint 2 deleted because the program has left the block in
which its expression is valid.
0x0000555555555a5b in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:131
131           Readiness r = assess_readiness(&fleet[i], &cfg, &blockers);
Value returned is $5 = R_BLOCKED


### 5. Break on the automatic-halt rule and test the canary boundary
Breakpoint 3 at 0x555555555505: file rollout_gate.c, line 76.


Breakpoint 3, should_halt (batch_index=0, batch_size=2, unhealthy=1, cfg=0x7fffffffe3b0) at rollout_gate.c:76
76        double frac = batch_size ? (double)unhealthy / batch_size : 0.0;
batch_index = 0
batch_size = 2
unhealthy = 1
cfg = 0x7fffffffe3b0
$6 = 0.10000000000000001
0x0000555555555baa in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:142
142        printf("halt(canary,2,1)=%d expect 1\n", should_halt(0, 2, 1, &cfg));
Value returned is $7 = 1


### 6. Evaluate the 10 percent tolerance boundary by hand
$8 = 0
$9 = 1
machines=120 mismatches=0
  ready                  42
  ready_with_caution     6
  needs_approval         4
  insufficient_evidence  14
  blocked                36
  compliant              18
halt(canary,2,1)=1 expect 1
halt(stage1,10,1)=0 expect 0 (10% is not > 10%)
halt(stage1,10,2)=1 expect 1
[Inferior 1 (process 461) exited normally]
```

## 9.4 What the session shows

- **Step 1–2:** a conditional breakpoint stops only on machine vm-003; the backtrace shows assess_readiness called from main, and *m shows driver = CHK_FAIL with telemetry present.

- **Step 3:** a hardware watchpoint on the blocker bit mask records its changes. The first reported change (Old value = 2 → 0) is the uninitialised stack slot being set by the declaration at line 50; the second (0 → 1) is the B1 bit being set when the failed mandatory check is processed at line 57.
- **Step 4:** finish returns R_BLOCKED, matching the Python verdict “blocked” for vm-003 (rule B1).
- **Step 5–6:** should_halt(batch 0, size 2, unhealthy 1) returns 1 (canary rule); evaluating 1/10 > 0.10 gives 0 and 2/10 > 0.10 gives 1, confirming the strict “greater than 10%” tolerance.

Limits of this evidence: it demonstrates that an independent native implementation of two rules agrees with the Python engine on this fleet and lets the logic be inspected in a debugger. It does not prove the Python code correct in general.

# 10. Screenshots (captured from the running application)

All screenshots were captured on 26 October 2026 from the application served at http://127.0.0.1:8000 on the Windows 11 host, after the page and its charts had finished rendering. Data are synthetic.

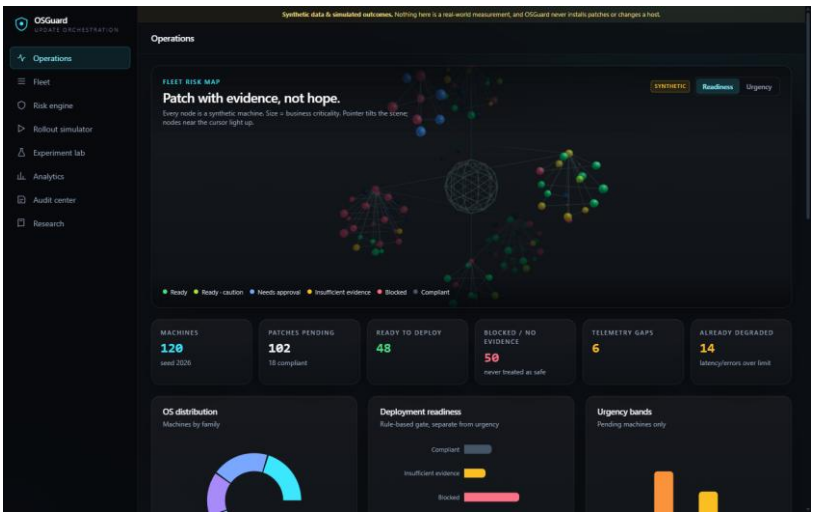



*Figure 2 – Operations dashboard with the interactive 3D fleet map (nodes coloured by readiness).*

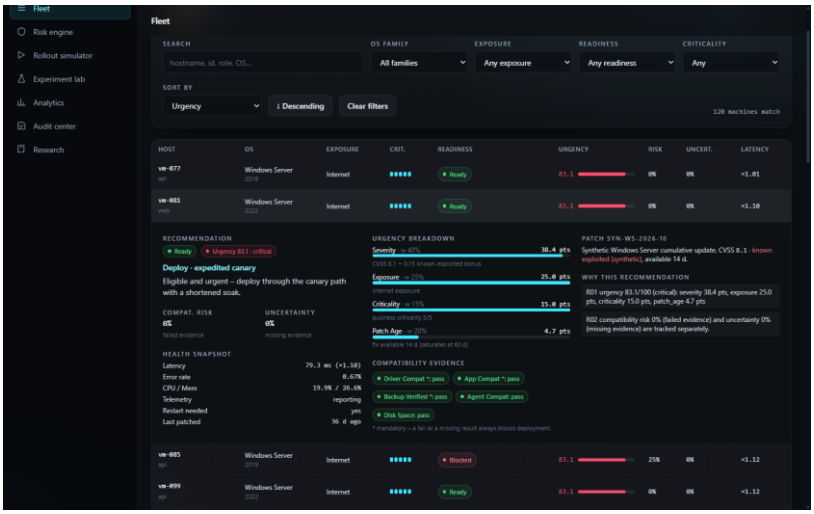



*Figure 3 – Fleet inventory with search/filter/sort and an expanded machine (vm-081): recommendation, urgency breakdown, evidence, health.*

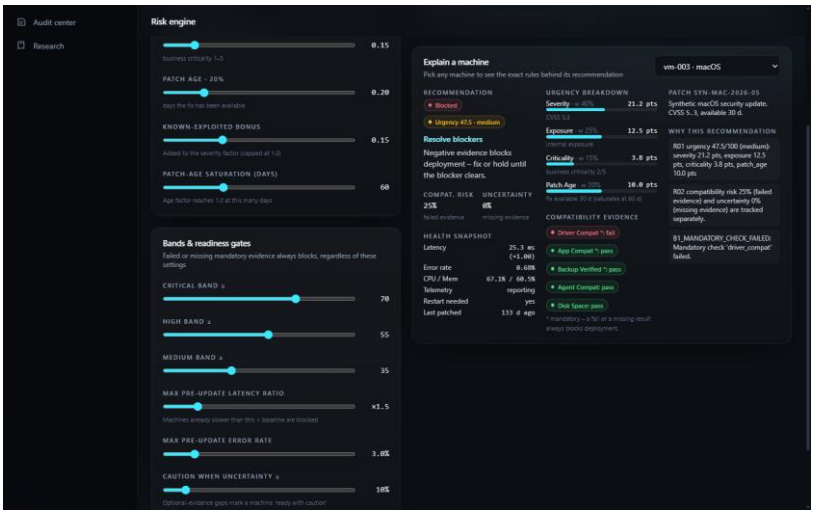



*Figure 4 – Risk engine: explanation of a blocked machine (vm-003) with rule B1 and separate risk/uncertainty.*

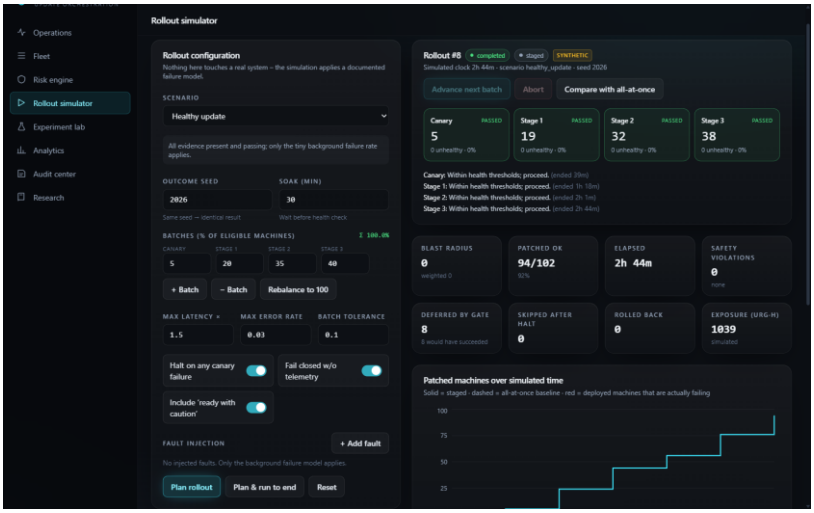



*Figure 5 – Rollout simulator: successful staged rollout, healthy-update scenario (all batches passed, 0 blast radius).*

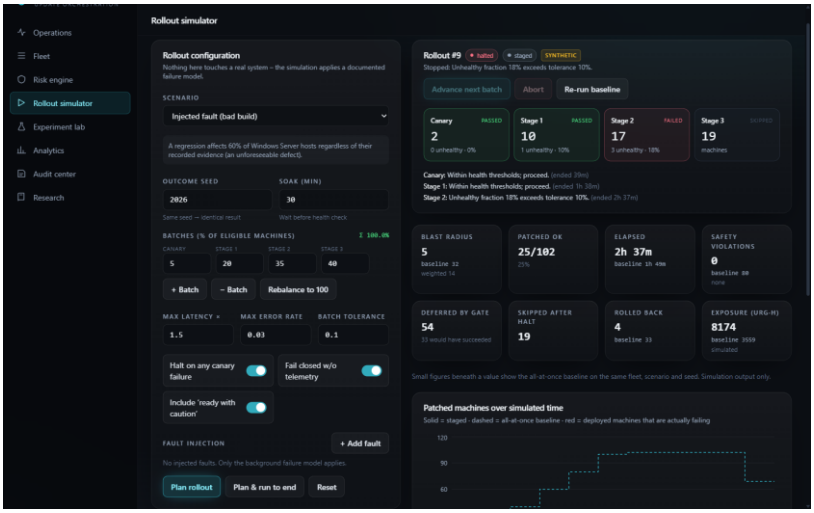



*Figure 6 – Rollout simulator: injected bad build; Stage 2 fails the 10% tolerance and the rollout halts, Stage 3 untouched; baseline figures alongside.*

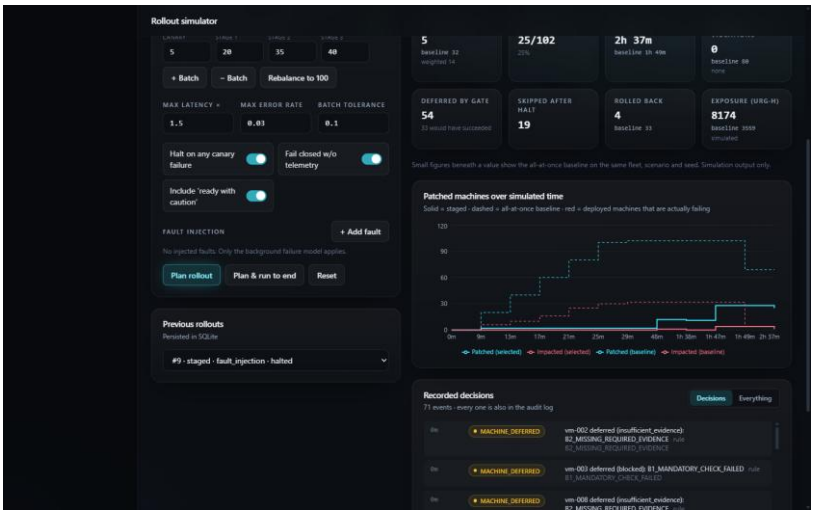



*Figure 7 – Patched and impacted machines over simulated time, staged (solid) versus all-at-once baseline (dashed).*




*Figure 8 – Analytics: staged versus all-at-once across scenarios, with the simulation disclaimer.*

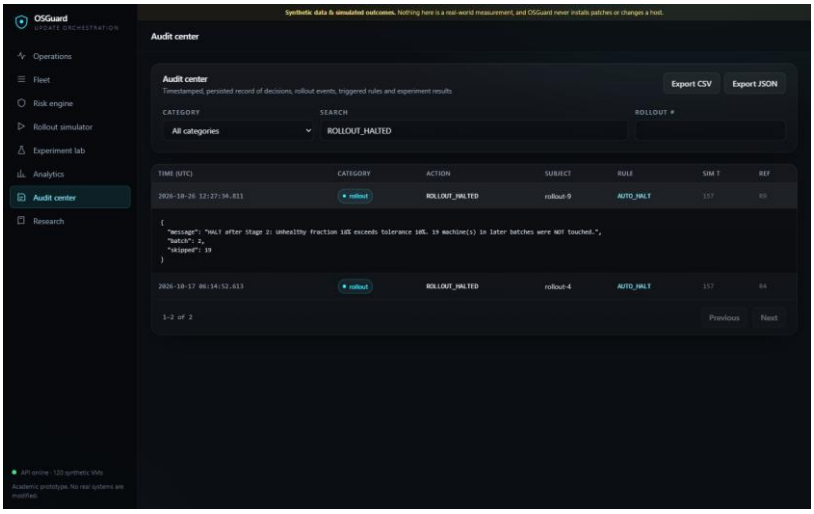



*Figure 9 – Audit center: persisted ROLLOUT_HALTED event with its rule (AUTO_HALT) and details.*

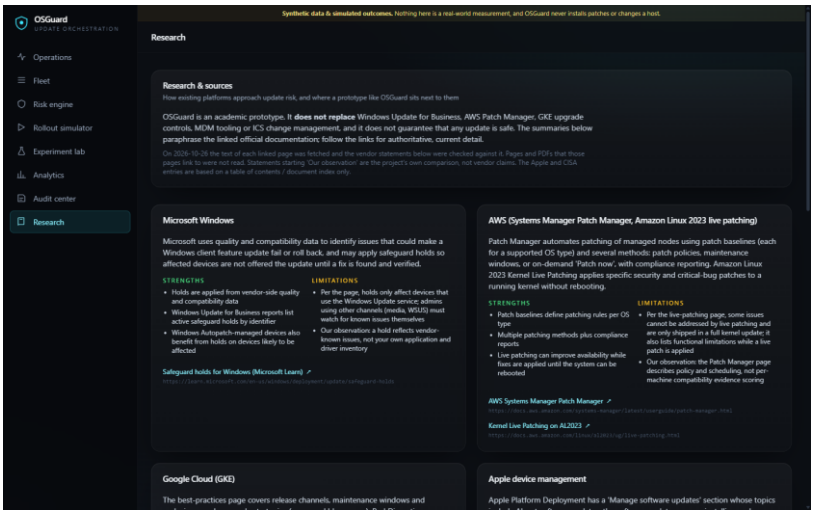



*Figure 10 – Research and sources page with the verified vendor summaries.*

# 11. Viva questions and answers

**Q1. Why are urgency, compatibility risk and uncertainty separate?** They answer different questions. A highly urgent machine can still be unsafe to patch, and an unknown result is not the same as a failed one. Blending them would hide why a machine was held and could let missing evidence look like safety.

**Q2. What happens when a mandatory check has no result?** Rule B2 makes the machine "insufficient evidence", so it is deferred from the staged rollout. Only negative evidence (B1, B4, B5) ranks higher (blocked). Missing evidence is never rounded up to ready.

**Q3. How is the canary chosen?** Eligible machines are ordered by criticality (lowest first); the canary is picked round-robin across OS families from that order so it covers several platforms without risking the most critical machines.

**Q4. What stops a bad update from spreading?** After each batch's soak the health check halts the rollout if the canary has any unhealthy machine or a batch's unhealthy fraction exceeds 10%. Unhealthy machines are rolled back and later batches are never touched.

**Q5. Why does the staged strategy show higher exposure?** Deferred and skipped machines stay unpatched for the horizon and batches take time. Safety is bought with exposure and delay; the application reports both openly.

**Q6. Is the benefit of staging proven?** No. The failure model correlates failures with recorded evidence, so gating helps by construction. The results demonstrate the mechanism and its trade-offs on synthetic data only.

**Q7. What is a subtle regression?** A failure that stays below the default health thresholds (latency 1.2–1.4× baseline, error rate 1.5–2.5%). It is not detected and is counted as undetected degradation; tightening thresholds can catch it.

**Q8. Why not machine learning?** The goal is explainability and reproducibility. A rule-based model can show exactly which rule fired; no learned model exists in the project and none is claimed.

**Q9. How are experiments reproducible?** The fleet comes from a fixed seed and each machine's hidden outcome is a SHA-256-derived value of (seed, machine, patch), independent of Python's hash randomisation. Same inputs give identical results.

**Q10. Where does GDB fit in, since the app is Python?** GDB only debugs the C diagnostic harness, which re-implements two safety rules and cross-checks the Python verdicts. It is not xv6, not a kernel and not the application.

**Q11. What failure did you find and fix?** An intermittent HTTP 500 caused by an SQLite connection used on a different thread than the one that opened it. Fixed with check_same_thread=False plus a concurrency regression test.

**Q12. How would it scale beyond 120 machines?** Planning and assessment are linear in machines and run in memory; the 500-machine API limit and single-file SQLite are the first limits. A real deployment would need a job queue, per-tenant storage, authentication and real telemetry; none is implemented.

**Q13. Why does this not replace Patch Manager, Windows Update or GKE tooling?** Those products actually deploy and enforce updates and use vendor data. OSGuard is a simulation that never touches a host and models decision logic only.

**Q14. What do the cited sources actually support?** Section 4 lists exactly what each page says; Apple and CISA were checked at index level only. Our own comparisons are labelled as observations.

# 12. Conclusion and future work

OSGuard shows, on a reproducible synthetic fleet, how separating urgency from failed and missing evidence, gating on explicit rules, and rolling out in health-checked stages can contain a bad update at the cost of delay and exposure. The application, tests, audit trail, experiments and a native cross-check were built and verified; the findings are limited to the simulation. Future work: model dependencies and maintenance windows, ingest real (consented) telemetry, calibrate the failure model against published incident data, add authentication and a job queue, and evaluate detection of subtle regressions.

# 13. References


1. Microsoft Learn – Safeguard holds for Windows. https://learn.microsoft.com/en-us/windows/deployment/update/safeguard-holds (checked 26 Oct 2026).
2. AWS – AWS Systems Manager Patch Manager. https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html (checked 26 Oct 2026).
3. AWS – Kernel Live Patching on AL2023. https://docs.aws.amazon.com/linux/al2023/ug/live-patching.html (checked 26 Oct 2026).
4. Google Cloud – GKE: best practices for upgrading clusters. https://docs.cloud.google.com/kubernetes-engine/docs/best-practices/upgrading-clusters (checked 26 Oct 2026).
5. Apple – Apple Platform Deployment. https://support.apple.com/guide/deployment/welcome/web (index checked 26 Oct 2026).
6. CISA – ICS Recommended Practices. https://www.cisa.gov/resources-tools/resources/ics-recommended-practices (index checked 26 Oct 2026).
7. Open-source software used: FastAPI, Pydantic, Uvicorn, pytest, React, Vite, Tailwind CSS, Framer Motion, Three.js, @react-three/fiber, Recharts, Vitest, GCC and GDB; each under its own licence. The VIT logo on the title page was supplied by the team.


# 14. Submission checklist and video package

*Table 17 – Checklist*

| Item | Status |
|---|---|
| PDF contains explanation, code, GDB log, screenshots | Done (this document) |
| Fill in editable title-page fields (course, lecturer, slot, date) | Team action |
| Add the lecturer's exact assignment questions and answers, if any were set | Team action |
| Record the walkthrough: every member speaks in their own voice, runs the program (script: docs/VIDEO_WALKTHROUGH_SCRIPT.md) | Team action |

| Item | Status |
|---|---|
| Upload to Microsoft Clipchamp or Google Drive and share with the evaluator's address given in the assignment (manoov.r@vit.ac.in in the text supplied), or YouTube with visibility Unlisted | Team action |
| Paste the video link on the title page and below | Team action |
| Fill the speaker timestamp table below after recording | Team action |
| Be ready for follow-up questions on your own implementation | Team action |

### Video link: [EDITABLE – paste after upload]

*Table 18 – Speaker allocation (segments follow docs/scripts; fill in timestamps after recording)*

| Team member | Suggested segment | Timestamp range |
|---|---|---|
| Anirudh Koppuravuri (24BCE0490) | Person 1 – introduction, OS used, architecture, dashboard and 3D map, fleet inventory | [mm:ss – mm:ss] |
| Kalivarapu Prabhas (24BCE2279) | Person 2 – scoring code, risk engine page, research sources, native C harness and GDB | [mm:ss – mm:ss] |
| Barath Sornaraj (24BCE2599) | Person 3 – simulator code and page, experiment lab, analytics, audit center, tests, conclusion | [mm:ss – mm:ss] |

# Appendix A. Source listings

Compact edition: key excerpts only. The edition named OSGuard_Project_Report_full_listings contains complete listings with hashes. Full files are in the OSGuard folder.

## A.1 scoring.py – urgency and readiness rules

```
def compute_urgency(machine: Machine, patch: Patch, cfg: ScoringConfig) -> Urgency:
    total_w = cfg.w_severity + cfg.w_exposure + cfg.w_criticality + cfg.w_age
    sev = min(1.0, patch.cvss / 10.0 + (cfg.exploited_bonus if patch.known_exploited else 0.0))
    exp = cfg.exposure_values[machine.exposure]
    crit = (machine.criticality - 1) / 4.0
    # Patch age = days the fix has been available but not applied.
    age =min(1.0, max(patch.released_days_ago, 0) / cfg.age_saturation_days)
    rows = [
        ("severity", sev, cfg.w_severity,
         f"CVSS {patch.cvss:.1f}" + (f" + {cfg.exploited_bonus:.2f} known-exploited bonus" if patch.known_exploited else "")),
        ("exposure", exp, cfg.w_exposure, f"{machine.exposure} exposure"),
        ("criticality", crit, cfg.w_criticality, f"business criticality {machine.criticality}/5"),
        ("patch_age", age, cfg.w_age,
         f"fix available {patch.released_days_ago} d (saturates at {cfg.age_saturation_days} d)"),
    ]
    comps = [
        Component(name=n, raw=round(r, 4), weight=round(w / total_w, 4), points=round(100 * r * w / total_w, 2), note=note)
        for n, r, w, note in rows
    ]
    score = round(sum(c.points for c in comps), 2)
    return Urgency(score=score, band=_band(score, cfg), components=comps)  # type: ignore[arg-type]


    blockers: list[Finding] = []
    warnings: list[Finding] = []
    negative: list[str] = []  # codes that are *negative* evidence
    missing: list[str] = []  # codes that are *absent* evidence


    for name in MANDATORY_CHECKS:
        st = machine.checks.get(name, "unknown")
        if st == "fail":
            blockers.append(Finding(code="B1_MANDATORY_CHECK_FAILED", message=f"Mandatory check '{name}' failed."))
            negative.append("B1")
        elif st == "unknown":
            blockers.append(Finding(code="B2_MISSING_REQUIRED_EVIDENCE",
                                    message=f"Mandatory check '{name}' has no result; absence of evidence is treated as NOT safe."))
            missing.append("B2")
    if cfg.require_telemetry and not machine.telemetry_present:
        blockers.append(Finding(code="B3_NO_TELEMETRY",
                                message="No health telemetry: post-update health cannot be verified, so rollout is not permitted."))
        missing.append("B3")
    if machine.os_version in patch.hold_versions:
        blockers.append(Finding(code="B4_VENDOR_HOLD", message=patch.hold_reason or "Vendor safeguard hold applies."))
        negative.append("B4")
    h = machine.health
    if h.latency_ratio > cfg.health_latency_ratio_max or h.error_rate > cfg.health_error_rate_max:
        blockers.append(Finding(
            code="B5_PRE_UPDATE_HEALTH_DEGRADED",
            message=f"Machine already unhealthy (latency x{h.latency_ratio:.2f} vs limit x{cfg.health_latency_ratio_max:.2f}, "
                    f"errors {h.error_rate:.1%} vs limit {cfg.health_error_rate_max:.1%}); a regression would be unattributable."))
        negative.append("B5")
    needs_approval = machine.os_family in cfg.manual_approval_families
    if needs_approval:
        blockers.append(Finding(code="B6_MANUAL_APPROVAL_REQUIRED",
                                message=f"{machine.os_family} changes require explicit human/vendor approval."))


    for name in OPTIONAL_CHECKS:
        st = machine.checks.get(name, "unknown")
        if st == "fail":
            warnings.append(Finding(code="W1_OPTIONAL_CHECK_FAILED", message=f"Optional check '{name}' failed."))
        elif st == "unknown":
            warnings.append(Finding(code="W2_OPTIONAL_CHECK_UNKNOWN", message=f"Optional check '{name}' has no result."))
    if not cfg.require_telemetry and not machine.telemetry_present:
        warnings.append(Finding(code="W3_TELEMETRY_OPTIONAL", message="Telemetry missing but policy does not require it; staged rollouts will
fail closed."))


    soft_blockers = [b for b in blockers if b.code == "B6_MANUAL_APPROVAL_REQUIRED"]
    hard = [b for b in blockers if b not in soft_blockers]


    # Precedence: negative evidence > missing evidence > approval gate > caution > ready
    if negative:
        readiness = "blocked"
    elif missing:
        readiness = "insufficient_evidence"
    elif hard:  # pragma: no cover - defensive; all hard blockers are classified above
        readiness = "blocked"
    elif soft_blockers:
        readiness = "needs_approval"
    elif warnings or uncertainty >= cfg.caution_uncertainty:
        readiness = "ready_with_caution"
    else:
        readiness = "ready"
```

## A.2 simulator.py – failure model and halt decision

```
def failure_model(m: Machine, patch: Patch, cfg: RolloutConfig, scoring: ScoringConfig, seed: int) -> tuple[bool, FaultKind | None]:
    """Return (fails, kind) – the simulation's hidden ground truth for this machine."""
    parts: list[tuple[float, FaultKind]] = [(cfg.base_failure_probability, "latency_regression")]
```

```
    if m.checks.get("driver_compat") == "fail":
        parts.append((P_DRIVER_FAIL, "boot_failure"))
    if m.checks.get("app_compat") == "fail":
        parts.append((P_APP_FAIL, "error_spike"))
    unknown = sum(1 for c in ("driver_compat", "app_compat", "backup_verified") if m.checks.get(c) == "unknown")
    if unknown:
        parts.append((1 - (1 - P_UNKNOWN_MANDATORY) ** unknown, "latency_regression"))
    h = m.health
    if h.latency_ratio > scoring.health_latency_ratio_max or h.error_rate > scoring.health_error_rate_max:
        parts.append((P_PRE_DEGRADED, "latency_regression"))
    if m.os_version in patch.hold_versions:
        parts.append((P_VENDOR_HOLD, "boot_failure"))
    for f in cfg.faults:
        if f.matches(m):
            parts.append((f.probability, f.kind))
    survive = 1.0
    for p, _ in parts:
        survive *= 1 - min(max(p, 0.0), 1.0)
    p_fail = min(0.97, 1 - survive)
    if unit(seed, "fail", m.id, patch.id) >= p_fail:
        return False, None
    kind = max(parts, key=lambda t: t[0])[1]
    if kind != "boot_failure" and unit(seed, "subtle", m.id) < P_SUBTLE:
        kind = "subtle_regression"
    return True, kind


    frac = len(unhealthy) / n if n else 0.0
    canary_hit = batch.index == 0 and cfg.canary_halt_on_any_failure and len(unhealthy) > 0
    breach = canary_hit or frac > cfg.max_unhealthy_fraction
    _emit(state, eval_at, "HEALTH_CHECK",
          f"{batch.label}: {len(unhealthy)}/{n} unhealthy ({frac:.0%}), {len(unverifiable)} unverifiable; "
          f"limits: latency x{cfg.max_latency_ratio}, errors {cfg.max_error_rate:.1%}, batch tolerance {cfg.max_unhealthy_fraction:.0%}.",
          batch=batch.index, rule="HEALTH_THRESHOLDS", unhealthy=len(unhealthy), unverifiable=len(unverifiable), fraction=round(frac, 4))
```

### A.3 native/rollout_gate.c – readiness and halt in C

```
Readiness assess_readiness(const Machine *m, const Config *cfg, int *blockers_out)
{
    int blockers = 0, negative = 0, missing = 0, warnings = 0;
    const Check mandatory[3] = { m->driver, m->app, m->backup };
    int i;


    if (m->compliant) { *blockers_out = 0; return R_COMPLIANT; }


    for (i = 0; i < 3; i++) {
        if (mandatory[i] == CHK_FAIL)         { blockers |= B1; negative = 1; }
        else if (mandatory[i] == CHK_UNKNOWN) { blockers |= B2; missing = 1; }
    }
    if (cfg->require_telemetry && !m->telemetry) { blockers |= B3; missing = 1; }
    if (m->vendor_hold)                          { blockers |= B4; negative = 1; }
    if (m->latency_ratio > cfg->latency_ratio_max || m->error_rate > cfg->error_rate_max) { blockers |= B5; negative = 1; }
    if (!strcmp(m->family, "ICS/Embedded"))      { blockers |= B6; }
    if (m->agent != CHK_PASS || m->disk != CHK_PASS) warnings = 1;


    *blockers_out = blockers;
    if (negative) return R_BLOCKED;                 /* negative evidence beats everything */
    if (missing)  return R_INSUFFICIENT;            /* missing evidence is never "safe" */
    if (blockers & B6) return R_NEEDS_APPROVAL;
    return warnings ? R_CAUTION : R_READY;
}


/* Returns 1 when the rollout must halt after evaluating a batch. */
int should_halt(int batch_index, int batch_size, int unhealthy, const Config *cfg)
{
    double frac = batch_size ? (double)unhealthy / batch_size : 0.0;
    int canary_hit = (batch_index == 0 && cfg->canary_halt_on_any_failure && unhealthy > 0);
    return canary_hit || frac > cfg->max_unhealthy_fraction;
}
```

# Appendix B. Start-up commands (Windows)

```
cd C:\Users\aniru\OneDrive\Documents\OpenMolBench\OSGuard
.\start.ps1                 # installs on first run, builds the UI, serves http://127.0.0.1:8000
.\start.ps1 -Dev            # API on :8000 + Vite dev server on http://localhost:5173
cd backend;  .\.venv\Scripts\python -m pytest -q
cd frontend; npm test; npm run build
```